Northbridge Guide

A campus information assistant with cited evidence

Gen AI and applications | ECSDI24402 | CCA Option 1

Report prepared 26 October 2026

Abstract. Northbridge Guide answers campus questions from 54 fictional records through a Streamlit interface. It retrieves relevant records, checks whether they support the question, and either requests a short hosted response or returns record text through a deterministic fallback. Both paths expose citations. An offline rerun on 40 prepared questions finds an expected record among the first five results for all 37 supported cases. All 40 answer-or-refuse decisions match the test labels. Mean reference-keyword coverage is 91.4%, and 38 cases meet every per-case condition. These results describe the local fallback over a prepared corpus. They do not measure hosted model reliability or performance on real student questions.

1 Problem and scope

A campus helpdesk question often depends on a precise detail, such as a registration window or the office that handles an exception. A plausible answer with the wrong date can be less useful than a refusal. The project therefore asks a bounded question. Can a small application find an answer in a known corpus and show the records behind it?

The prototype supports questions about campus policies and services. It has no connection to a university database, student account, or live notice feed. It cannot check an individual's eligibility or confirm that a policy is current. The project demonstrates retrieval-augmented generation, or RAG, by giving the model selected evidence before it writes an answer.

The implementation has four practical objectives. It keeps source records inspectable, retrieves direct and paraphrased questions, refuses questions with weak support, and measures the resulting behavior with a repeatable test set.

2 Dataset and preparation

The knowledge base is `data/campus_records.jsonl`. It contains six records in each of nine categories. These are academics, examinations, fees, library, scholarships, clubs and societies, placements and careers, facilities and IT, and safety and wellbeing.

Each JSONL line stores an ID, category, title, content, source label, effective date, tags, confidence marker, and source type. The confidence value is `curated-synthetic`. It identifies the record's origin rather than a calibrated probability. Short records act as the retrieval units, so there is no PDF extraction or automatic text-chunking stage.

The loader validates record fields and rejects duplicate IDs. The dataset validation command also checks that evaluation references point to existing records. Effective dates travel with the evidence, but the code does not automatically expire records or resolve conflicting policy versions.

Data boundary. Northbridge is fictional. Its policy dates, amounts, source labels, and contacts are classroom material. The records must not guide real campus decisions.

Evidence and reproduction. Sources are `data/campus_records.jsonl`, `data/evaluation_set.jsonl`, `src/campus_guide/`, and `app.py`. Run these commands from the project root.

```
python scripts/validate_dataset.py
python -m unittest discover -s tests
python scripts/evaluate.py -output tmp/report_evaluation.json
```

3 Retrieval and answer policy

3.1 Ranking records

The index builds its search text from each record's title, content, category, and tags. It repeats titles and tags to give those short descriptive fields more weight. Tokenization lowercases text, removes common stop words, and normalizes a few phrases such as *check out*.

For the default index, the score for question q and record d is

$$S(q, d) = 0.52\, L(q, d) + 0.23\, C(q, d) + 0.25\, G(q, d).$$

Here L is cosine similarity between term-frequency vectors weighted by inverse document frequency. G is cosine similarity between character-trigram counts. C measures the overlap between expanded question terms and record terms, divided by the geometric mean of their set sizes. These are hand-set weights in `retrieval.py`, not learned parameters.

The concept table groups terms such as *exam* and *examination*, or *fee* and *installment*. Character trigrams provide another signal for spelling and fragment overlap. The default system therefore combines lexical signals and a curated synonym mechanism. It does not use learned semantic embeddings.

When explicitly enabled, the optional `sentence-transformers` path adds dense similarity. Its weights are 0.40 for lexical similarity, 0.18 for concept overlap, 0.17 for character similarity, and 0.25 for dense similarity. If the package or model cannot load, retrieval retains the default scoring path. This optional mode was not used for the reported results.

3.2 Deciding whether evidence is enough

A related record can still fail to answer the question. A placement-process record, for example, does not establish a placement salary. The implementation checks terms that carry specific meaning, including *package*, *current*, *phone*, and *refund*.

After sorting, the index can promote a record that covers all recognized concept groups and the required precision terms. The answer service then checks the leading record. Missing precision terms or incomplete concept-group coverage cause refusal. It also refuses when direct term coverage is below 0.18 and the unsupported-term ratio exceeds 0.50. A separate score check rejects a result below the default 0.16 threshold when direct coverage is below 0.20.

These conditions are heuristics. A score of 0.16 is a retrieval cutoff, not a 16% probability of correctness. The service filters evidence records again before constructing the answer context.

3.3 Two answer paths

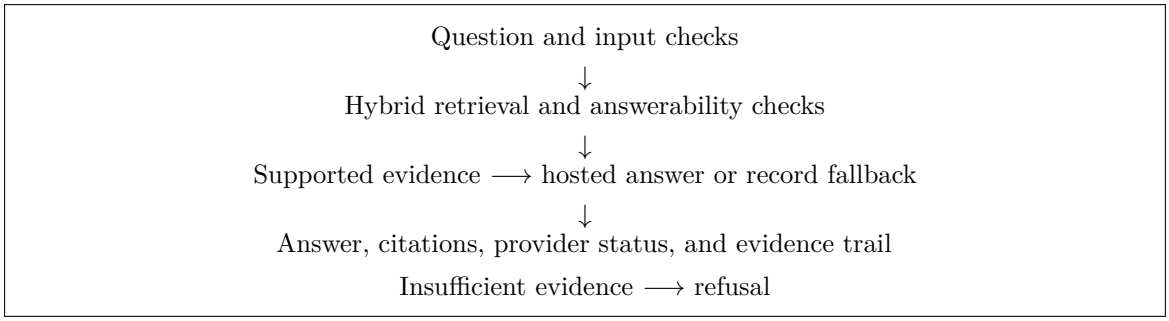
The normal answer service considers at most four context records. A configured provider sends the question and labeled evidence to an OpenRouter chat-completions endpoint. The repository's default model setting is `openrouter/free`. The prompt asks for evidence-only answers under 120 words and labels such as `[R1]`.

The service removes labels outside the supplied evidence. It accepts a hosted answer only if it contains a valid citation, nonempty text, and no more than 120 words after citation removal. Otherwise, or after a provider error, it returns the leading record's content. The fallback can append a second compatible record when its score is at least 82% of the leading score.

Citation validation checks that a label refers to supplied evidence. It does not check whether each generated claim follows from that evidence. That distinction limits what the hosted path can guarantee.

4 Application and implementation

The application uses Python 3.10 or newer. Streamlit supplies the browser interface, and `requests` handles hosted API calls. The default retrieval code uses Python's standard library. No model training, fine-tuning, vector database, or GPU is required for the local fallback.



Component	Responsibility
`models.py`, `data.py`	Record structures, JSONL loading, and validation.
`retrieval.py`	Search vectors, ranking, and support signals.
`answering.py`	Refusal decisions, evidence context, citations, and fallback.
`providers.py`, `settings.py`	Hosted requests and configuration.
`evaluation.py`	Per-case checks and aggregate metrics.
`app.py`	Streamlit views and browser-session state.

4.1 What the user can inspect

The **Ask** view presents the answer with its provider path, retrieval metadata, and evidence trail. A user can inspect the underlying source text rather than rely on the response alone. The **Evaluation** view runs the local baseline and shows per-case results. The **Corpus** view lets users filter and read the prepared records.

Blank questions and questions longer than 800 characters receive input refusals. The interface caps requests at 30 per browser session by default. Model answer text renders as Markdown with raw HTML disabled. The browser-session cap can limit a demonstration session, but it is not an account-wide quota or abuse-control mechanism.

4.2 Examples from the evaluation corpus

The question *When can I register for Term 1 courses?* points to record `acad-002`. That fictional record gives a registration window of 22 June to 3 July 2026 and directs students who miss it to their programme office. A fallback answer returns this record text with a citation.

The refusal cases ask about a current rector, a cafeteria menu, and placement-package outcomes. The corpus lacks those answers. The expected behavior is a refusal, even if retrieval finds another record in a related category.

4.3 Local execution

The repository documents `uv sync -dev` to install the project environment and `uv run streamlit run app.py` to launch the interface. Hosted generation requires provider configuration. The fallback remains usable without an API key, which makes the classroom demonstration independent of hosted model availability.

5 Evaluation and findings

The report rerun on 26 October 2026 validated both JSONL files, passed all 22 unit tests, and executed the 40-case offline evaluation. The evaluation script constructs the answer service with `provider=None`, so no hosted model contributes to these results. Its retrieval check searches five records separately from the answer service's four-record context limit.

Measure	Result	What the implementation counts
Retrieval hit rate at 5	37/37	At least one expected record occurs in the first five results for each supported case.
Refusal accuracy	40/40	The answer-or-refuse decision matches the expected flag.
Citation validity	37/37	Each supported answer cites at least one record, and every cited ID belongs to its evidence list.
Mean keyword coverage	91.4%	Mean fraction of reference-answer keywords found in supported answers.
Overall case pass rate	38/40	All retrieval, refusal, citation, and keyword conditions pass.

The code names the first metric recall at k. It is more precisely a hit rate because one expected record is sufficient even when a case lists several. Keyword coverage uses sets of lowercased tokens longer than two characters after stop-word removal. It does not measure sentence meaning or verify numbers against an external source.

Two supported cases miss the 75% keyword threshold. Course registration, `eval-02`, scores 70.0%. The safety case, `eval-34`, scores 71.4%. Both retrieve an expected record and pass the refusal and citation checks. The registration example contains the required dates, but differences such as *register* and *registration* still affect literal token overlap. These failures show why keyword coverage alone cannot establish answer quality.

6 Limits and next steps

The test set is small and prepared around the same fictional corpus as the application. Three refusal cases provide limited evidence about unsupported questions. The perfect retrieval and refusal scores do not establish performance on unseen student phrasing, mixed-topic questions, conflicting records, or hostile prompts.

The rerun does not evaluate hosted generation, dense retrieval, concurrent users, or the live browser interface. Unit tests cover local behavior, but their success does not establish production reliability. The current heuristics may accept a partially supported question or refuse a valid paraphrase.

A stronger evaluation would use independently written questions, more refusal cases, and human judgments of factual support. It would compare plain lexical retrieval with the hybrid and dense modes. Hosted answers need separate claim-level checks. A real deployment would also need approved policy sources, named content owners, version and expiry rules, privacy controls, and a route to a human helpdesk.

7 Conclusion

Northbridge Guide implements an inspectable question-to-evidence workflow in a small application. The verified fallback finds expected records for all supported test questions, while two cases miss the project's keyword threshold. Its useful result is a reproducible baseline with visible source records and explicit refusals. The present evidence supports a classroom demonstration, not an official campus advice service.